

# **Data Engineering and Machine Learning Pipeline**

## **Phase II Project Report**

**ENGE707 Data Engineering and AI**

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

# Table of Content

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>1. Introduction – Project Phase-I Summary.....</b>                                 | <b>4</b>  |
| 1.1 Problem Restatement.....                                                          | 5         |
| 1.2 Dataset summary.....                                                              | 5         |
| 1.3 Data-quality issues identified in Phase I.....                                    | 6         |
| 1.4 Justification for an LCS-based solution.....                                      | 6         |
| <b>2. Original LCS System on Raw Dataset.....</b>                                     | <b>7</b>  |
| 2.1 LCS variant selection (preferably eLCS) – justification.....                      | 7         |
| 2.2 Confirmation that original LCS code is used without algorithmic modification..... | 8         |
| 2.3 Execution on raw / minimally processed dataset.....                               | 9         |
| 2.4 LCS parameter settings used.....                                                  | 11        |
| 2.5 Baseline performance results.....                                                 | 12        |
| 2.6 Minimal processing required for LCS compatibility.....                            | 13        |
| <b>3. Data Preprocessing and Feature Engineering.....</b>                             | <b>14</b> |
| 3.1 Data-leakage prevention measures.....                                             | 14        |
| 3.2 Data cleaning.....                                                                | 15        |
| 3.3 Outlier investigation and handling.....                                           | 18        |
| 3.4 Feature engineering.....                                                          | 19        |
| 3.5 Encoding categorical variables.....                                               | 20        |
| 3.6 Discretising & Encoding numerical variables.....                                  | 20        |
| 3.7 Class-imbalance handling.....                                                     | 21        |
| 3.8 Expected impact of preprocessing on LCS performance.....                          | 22        |
| <b>4. Improved LCS-Based System.....</b>                                              | <b>22</b> |
| 4.1 Summary of changes vs. the original LCS system.....                               | 23        |
| 4.2 Justification – expected effect on accuracy or model quality.....                 | 23        |
| 4.3 Algorithmic explanation (pseudocode or flowchart).....                            | 23        |
| 4.4 Comparison against the original LCS baseline.....                                 | 23        |
| <b>5. Experiments.....</b>                                                            | <b>23</b> |
| 5.1 Validation strategy (80/20 split or k-fold CV) – justification.....               | 24        |

|                                                                                                                                                                                                                                         |           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 5.2 Evaluation metrics – selection and justification (classification: accuracy, precision, recall, F1, balanced accuracy, ROC-AUC, PR-AUC, confusion matrix; regression: MAE, MSE, RMSE, MAPE, R <sup>2</sup> , residual analysis)..... | 24        |
| 5.3 Statistical significance testing.....                                                                                                                                                                                               | 24        |
| 5.4 Interpretation of significance results.....                                                                                                                                                                                         | 24        |
| <b>6. Model Comparison.....</b>                                                                                                                                                                                                         | <b>24</b> |
| 6.1 Original LCS – raw/minimally processed data.....                                                                                                                                                                                    | 25        |
| 6.2 Original LCS – preprocessed data.....                                                                                                                                                                                               | 25        |
| 6.3 Improved LCS – preprocessed/feature-engineered data.....                                                                                                                                                                            | 25        |
| 6.4 At least three conventional ML models (e.g. Random Forest, SVM, Decision Tree, Logistic Regression, Naive Bayes, k-NN, Gaussian Process) – no deep learning.....                                                                    | 25        |
| 6.5 Comparative results table.....                                                                                                                                                                                                      | 25        |
| <b>7. Interpretation of Results and Explainable AI.....</b>                                                                                                                                                                             | <b>25</b> |
| 7.1 Interpretation of main findings from the model comparison.....                                                                                                                                                                      | 26        |
| 7.2 Explanation of key LCS rules / feature conditions / classifier patterns.....                                                                                                                                                        | 26        |
| 7.3 At least three sample LCS rules discussed for decision-making insight.....                                                                                                                                                          | 26        |
| 7.4 How rule-based outputs support interpretability.....                                                                                                                                                                                | 26        |
| 7.5 Trustworthiness and practical meaningfulness of the explanations.....                                                                                                                                                               | 26        |
| <b>8. Discussion, Limitations, and Future Work.....</b>                                                                                                                                                                                 | <b>26</b> |
| 8.1 Reasons for performance improvement (or lack thereof).....                                                                                                                                                                          | 26        |
| 8.2 Practical significance of results beyond raw numbers.....                                                                                                                                                                           | 26        |
| 8.3 Limitations – data quality, dataset size, class imbalance, noise, missingness, feature relevance...26                                                                                                                               |           |
| 8.4 Future work – improved LCS variants, better preprocessing, external validation, domain expert review.....                                                                                                                           | 26        |
| <b>References.....</b>                                                                                                                                                                                                                  | <b>27</b> |

# 1. Introduction – Project Phase-I Summary

- ☐ Students must briefly summarise the Phase I project. Students are required to:
  - ☒ ~~Restate the selected problem, dataset, target variable, and prediction objective.~~
  - ☒ ~~Summarise the dataset source, number of records, number of features, and main data characteristics.~~
  - ☐ Briefly describe the main data-quality issues identified in Phase I.
  - ☐ Explain why the problem is suitable for an LCS-based solution.

## Link with justification, do not repeat

- raw data, no preprocessing
- base elcs model
- apply preprocessing, we want to see if our preprocessing was good or not
  - Evidence between pre processing and the processed data
  - reducing the search space -
- Make some changes in the ELCS base system to improve the accuracy or computational cost  
e.g. lowering the training amount (Novelty)
  - changing fitness function is novelty
- Training/Testing sets should be run at least 10 times. take the average of 10 times
  - use different seed
  - *also mention the seeds in the report.*
- cross fold validation not random and it is in sequence
- **don't forget to randomize when separating testing and training data**

## 1.1 Problem Restatement

Heart disease is a major concern that currently affects 195,000 people in New Zealand alone. Approximately every 90 minutes, a New Zealander dies from heart disease, and it is the number one killer for women (*Latest Heart Disease Statistics*, n.d.). In this project, the dataset “281K Patient Dataset for Multi-Disease Prediction” from Kaggle is used. The goal is to find whether the lifestyle, demographics, and clinical measurements in the dataset have any relevance to predicting whether an individual has heart disease. A model with this prediction ability isn’t a full replacement for having an in-person cardiovascular risk assessment. Still, it could be a cost-effective way to check for an individual’s chance of having heart disease. The target column “heart\_disease\_target” is a binary classification created after cleaning and is based on the “sublabel” column, which contains the combination of diseases an instance has. If the instance from the sublabel contains “HT”, the value of the new column would be True; otherwise, False. This new column is created to be easily used to compare and train the machine learning model’s prediction capabilities.

## 1.2 Dataset summary

The dataset is a combination of three datasets (diabetes, heart disease, and hypertension). There are 280,985 raw records, and 278,701 remain after removing duplicates; the 278,701 will be used instead of the raw amount. The original dataset has 39 columns, but the introduction of “heart\_disease\_target” gives 40 columns. The columns utilised during the analysis and processing stage are lifestyle, demographics, clinical, and disease indicators. No missing values were found during the dataset inspection.

The overall heart disease rate is 2.1% across all these sources, but individually the dataset shows varying heart disease rates: 20.06% in the heart disease dataset source, compared to 4.17% for the diabetes source, and 0% for the hypertension source.

### 1.3 Data-quality issues identified in Phase I

From the data inspection and data cleaning stage, multiple issues in the dataset were identified:

- 2,284 duplicate instances were removed.
- heart\_disease has fractional values (from 0 to 1), which doesn't clearly explain whether an individual has heart disease, so a new column (heart\_disease\_target) was created to serve as the target (binary) instead of the heart\_disease column.
- The columns sublabel, disease\_flags, label, and heart\_disease all contain data leakage information. This provides clues and partial answers to whether the individual has heart disease; thus, they are excluded from the prediction process.
- Some columns such as age/age\_level/age\_normalized and bmi/bmi\_level carry the same information; this is to be reviewed before modelling.
- Imbalance in the distribution of instances with and without heart disease, since 97.9% of records do not have heart disease and 2.1% do.
- As described in the dataset summary, individual sources have different heart disease rates.

### 1.4 Justification for an LCS-based solution

From the exploratory data analysis in phase 1, the heat map demonstrated that there was a weak linear correlation with the heart\_disease\_target; e.g age is 0.08, BMI is 0.04, and triglycerides are 0.09. This suggests that a single feature alone does not have a strong correlation with the target. When just analysing BMI level and age group, a clear difference in heart disease rate was observed between different age groups and BMI levels; seniors who are also overweight displayed the highest heart disease rate of 6.53% compared to 2.1% of the overall heart disease rate. Adding gender results in a difference in heart disease rates of 8.18% for males and 4.99% for females who are seniors and overweight. The overall takeaway is that single features are weak as linear predictors, and that combining specific features together showed a greater heart disease rate.

The main reason why LCS is suitable for this problem is that it allows us to find a specific set of conditions that predicts the heart\_disease\_target well. In the exploratory data analysis, all possible combinations of features with the target could not be examined due to time constraints as well as the number of possible combinations of features in the dataset; therefore, LCS would help with automation. Another reason is that LCS involves a population to create if-then rules (classifiers). This could allow interpretability for each classifier, as the produced model aims to help predict heart disease. Experts in the field could analyse these attributes to see whether each classifier makes medical sense. However, there is a challenge: the dataset has a clear imbalance of instances with heart disease compared to those without; thus, the LCS machine learning model may favour the majority class. Further evaluation would be needed beyond just the accuracy.

## 2. Original LCS System on Raw Dataset

- ☒ ~~Students must run an original LCS system on the raw or original dataset before making improvements. Students are required to:~~
  - ☒ ~~Select an appropriate LCS variant, preferably eLCS.~~
  - ☒ ~~Use the original LCS code without algorithmic modification.~~
  - ☒ ~~Run the original LCS system on the raw or minimally processed dataset.~~
  - ☒ ~~Report the selected LCS parameters.~~
  - ☒ ~~Record the baseline performance results.~~
  - ☒ ~~Explain any minimal processing required to make the dataset compatible with the LCS code.~~

### 2.1 LCS variant selection (preferably eLCS) – justification

We used eLCS, also known as Education Learning Classifier System, as our LCS system for this project. eLCS is a variant of Learning Classifier systems that uses supervised learning and is used for classification problems. This project's goal is to successfully predict whether a patient has heart disease, which is represented by "heart\_disease\_target" with a value of True or False. A True/False value can be represented by binary 1 for True and 0 for False, making it a binary classification problem.

In our dataset, “281K patient dataset”, there are both continuous and categorical variables. eLCS can work with both of these types of variables, which made it a suitable LCS for this project.

We learned in phase I that there were no relationships between individual features and the target attribute. However, after combining features together and then comparing them, the relationships were clearer. eLCS is capable of learning rules that include a combination of attributes, which is exactly what this project requires for a machine learning model.

## 2.2 Confirmation that original LCS code is used without algorithmic modification

```
setup(  
    name = 'scikit-eLCS',  
    packages = ['skeLCS'],  
    version = '1.2.4',  
    license='License :: OSI Approved :: GNU General Public License v3 (GPLv3)',  
    description = 'Educational Learning Classifier System',
```

**Figure 2.2.1: eLCS version confirmation**

Our original dataset was given to the original eLCS implementation. The version used was 1.2.4, and there were no modifications made to the source code or behaviour of the system. The setup remained untouched and was found in the “setup.py” file in the system files. External code was added outside the eLCS implementation package, but this was just to make the dataset ready for model training. The following code was added:

1. Loading the dataset
2. Creating the target attribute
3. Splitting the dataset into training and testing sets (80/20 split)
4. Predicting the results
5. Calculating all of the metrics from model training

## 2.3 Execution on raw / minimally processed dataset

The raw dataset from “patient\_data.csv” was used for the baseline eLCS model. As shown in Figure 2.3.1, there were 280,985 records and 39 rows. The raw dataset contained 2284 duplicate records, which were intentionally kept. The duplicate rows will be handled during Task 3 because the original eLCS model was meant to represent the raw dataset with none to very minimal preprocessing.

```
Raw shape: (280985, 39)
Raw records: 280985
Raw columns: 39
Missing values: 0
Duplicate records: 2284
```

Figure 2.3.1:

### Raw Dataset

```
heart_disease_target
0    275081
1     5904
Name: count, dtype: int64
```

| heart_disease_target | 0      | 1    |
|----------------------|--------|------|
| sublabel             |        |      |
| DI                   | 31918  | 0    |
| DI_HT                | 0      | 1398 |
| DI_HT_HY             | 0      | 845  |
| DI_HY                | 66473  | 0    |
| HT                   | 0      | 2595 |
| HT_HY                | 0      | 1066 |
| HY                   | 69806  | 0    |
| N                    | 106884 | 0    |

The target attribute was not in the original dataset, so it was added after loading. The “sublabel” attribute was used to create it by checking if the value contained “HT”, which represented heart disease. The values including “HT” were mapped to class 1, while the values not including it were mapped to class 0.

The result of the target creation was that 275,081 records were mapped to class 0 (no heart disease), and 5,904 records were mapped to class 1 (heart disease). With these results, the positive class rate was approximately 2.10%.

Figure 2.3.2: Target Construction

After the target attribute was added, there were a total of 40 columns. However, 4 of these were excluded from the predictor matrix due to having a one-to-one correlation

```
Predictor shape: (280985, 36)
Target shape: (280985,)
Positive-class rate: 0.021011797782799792
```

Figure 2.3.3: Predictor Set

with the target. Therefore, the baseline experiment used 36 columns. We performed validation checks which confirmed the target attribute was binary and the dataset contained no missing values.

The dataset was split into training and testing sets with an 80/20 split. Stratification was used to preserve the class imbalance in both sets so that they could both be as close to the original dataset split as possible.

```
Training shape: (224788, 36)
Testing shape: (56197, 36)
Training positive rate: 0.021010908055590156
Testing positive rate: 0.021015356691638343
```

**Figure 2.3.4: 80/20 Split results**

The results shown in Figure 2.3.4 tell us that the training set contained 224,788 rows (80% of rows from the dataset) and the testing set contained 56197 rows (20% of the dataset). Both sets used 36 attributes because 4 columns from the original dataset had a one-to-one correlation with the target attribute. Both sets maintained the same positive rate of approximately 2.10%, which was the same rate in the raw dataset.

```
Starting seed 3..
Completed seed 3: balanced accuracy=0.5000, ROC-AUC=0.6925, PR-AUC=0.2241, positive predictions=0
Starting seed 6..
Completed seed 6: balanced accuracy=0.5000, ROC-AUC=0.7354, PR-AUC=0.1711, positive predictions=0
Starting seed 9..
Completed seed 9: balanced accuracy=0.5000, ROC-AUC=0.6086, PR-AUC=0.0782, positive predictions=0
Starting seed 12..
Completed seed 12: balanced accuracy=0.5000, ROC-AUC=0.6885, PR-AUC=0.1787, positive predictions=0
Starting seed 15..
Completed seed 15: balanced accuracy=0.5000, ROC-AUC=0.7252, PR-AUC=0.2188, positive predictions=0
Starting seed 18..
Completed seed 18: balanced accuracy=0.5000, ROC-AUC=0.6970, PR-AUC=0.2439, positive predictions=0
Starting seed 21..
Completed seed 21: balanced accuracy=0.5000, ROC-AUC=0.6830, PR-AUC=0.1565, positive predictions=0
Starting seed 24..
Completed seed 24: balanced accuracy=0.5000, ROC-AUC=0.7186, PR-AUC=0.1668, positive predictions=0
Starting seed 27..
Completed seed 27: balanced accuracy=0.5000, ROC-AUC=0.6563, PR-AUC=0.2024, positive predictions=0
Starting seed 30..
Completed seed 30: balanced accuracy=0.5000, ROC-AUC=0.6630, PR-AUC=0.1845, positive predictions=0
```

**Figure 2.3.5: Results of the 10 baseline experiments**

Figure 2.3.5 shows the results of the 10 repeated experiments using the original eLCS model. The seeds used were [3, 6, 9, 12, 15, 18, 21, 24, 27, 30], verifying that each experiment was unique. In every run, the seed affected both the shuffled split (pseudo-random number generator) and the random state of the eLCS. Besides the seed, all of the parameters were kept the same throughout all 10 experiments. We will use the same seeds in Part 4 for the improved eLCS model to make the comparison between data fair.

## 2.4 LCS parameter settings used

|    | Parameter                  | Value      |
|----|----------------------------|------------|
| 0  | N                          | 1000       |
| 1  | acc_sub                    | 0.99       |
| 2  | beta                       | 0.2        |
| 3  | chi                        | 0.8        |
| 4  | delta                      | 0.1        |
| 5  | discrete_attribute_limit   | 10         |
| 6  | do_GA_subsumption          | True       |
| 7  | do_correct_set_subsumption | False      |
| 8  | fitness_reduction          | 0.1        |
| 9  | init_fit                   | 0.01       |
| 10 | learning_iterations        | 10000      |
| 11 | match_for_missingness      | False      |
| 12 | mu                         | 0.04       |
| 13 | nu                         | 5          |
| 14 | p_spec                     | 0.5        |
| 15 | random_state               | 3          |
| 16 | reboot_filename            | None       |
| 17 | selection_method           | tournament |
| 18 | specified_attributes       | []         |
| 19 | theta_GA                   | 25         |
| 20 | theta_del                  | 20         |
| 21 | theta_sel                  | 0.5        |
| 22 | theta_sub                  | 20         |
| 23 | track_accuracy_while_fit   | False      |

Figure 2.4.1: Parameters List

When creating the baseline model, we used the seed as the random\_state. So all **parameters were kept the same except for the seed** for each baseline experiment.

The parameters in Figure 2.4.1 were retrieved using “model.get\_params()”, which listed all of the parameters from the newly created eLCS object. Each of these affects how the experiment runs; for example, “learning\_iterations” represents how many times eLCS performs learning iterations, “N” represents the maximum rule-population size, and “mu” represents the crossover probability. The rest of the explanations are in “Original\_eLCS\_Baseline.ipynb” in the project files.

```
# 10 reproducible seeds for the repeated experiments.  
SEEDS = [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]  
✓ 0.0s
```

Figure 2.4.2: Seeds list

We used 10 different seeds (Figure 2.4.2) in **10 different and unrelated baseline experiments**. For each seed, a new eLCS model object was created, where the seed affected the train-test set split and the randomness of the experiment. The seed used for each experiment was recorded in order to make each run reproducible. As shown in Figure 2.3.5, the seed was recorded along with the experiment results for each run.

## 2.5 Baseline performance results

|   | Metric               | mean      | std          | min       | max       |
|---|----------------------|-----------|--------------|-----------|-----------|
| 0 | Accuracy             | 0.978985  | 1.170278e-16 | 0.978985  | 0.978985  |
| 1 | Precision            | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 2 | Recall               | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 3 | F1                   | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 4 | Balanced accuracy    | 0.500000  | 0.000000e+00 | 0.500000  | 0.500000  |
| 5 | ROC-AUC              | 0.686804  | 3.743284e-02 | 0.608623  | 0.735379  |
| 6 | PR-AUC               | 0.182506  | 4.613999e-02 | 0.078235  | 0.243927  |
| 7 | Positive predictions | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 8 | Training seconds     | 13.091345 | 1.869238e-01 | 12.765835 | 13.357953 |

**Figure 2.5.1: Baseline eLCS results after runs**

The accuracy from the baseline experiment was approximately 97.9%. However, this is only because many records belong to class 0. If the model predicts every record as class 0, then the final accuracy will be 97.9%. Due to this, the value from the result does not give us the true model accuracy we are looking for. The precision, recall, and F1 score were all zero.

Precision and recall both rely on positive heart-disease cases being predicted. And the F1 score combines them both. Because there were no positive heart-disease predictions, these scores cannot be calculated and are therefore zero. The balanced accuracy was 0.50 according to Figure 2.5.1. This means the model has chance-level performance, telling us that our model is the same as guessing. This is caused when the model only predicts one class and never the other.

The ROC-AUC and PR-AUC look at the probability scores. There were patients who received higher scores, but because they weren't high enough, they weren't predicted as having heart disease. The 0.687 value of ROC-AUC means that the model can separate the two classes, but it is not strong enough to cause positive predictions. PR\_AUC is more useful for this experiment because of the imbalanced dataset. The value is 0.183 for PR\_AUC, which means that the model's probability scores ranked heart-disease cases better than random. Even though this checks the relationship between precision and recall, which were both zero, it only needs the scores, which is why PR\_AUC isn't also zero.

|   | Confusion category | mean    | std | min     | max     |
|---|--------------------|---------|-----|---------|---------|
| 0 | True negatives     | 55016.0 | 0.0 | 55016.0 | 55016.0 |
| 1 | False positives    | 0.0     | 0.0 | 0.0     | 0.0     |
| 2 | False negatives    | 1181.0  | 0.0 | 1181.0  | 1181.0  |
| 3 | True positives     | 0.0     | 0.0 | 0.0     | 0.0     |

**Figure 2.5.2: Confusion Category Summary**

According to Figure 2.5.2, all experiments correctly predicted 55,016 class 0 cases, and incorrectly predicted 0 class 1 cases. There were no false positives or true positives, meaning the model never predicted

class one, and that no heart disease cases were detected. Zero false positives is usually a good thing,

but in this case, it didn't predict heart disease at all, meaning that it missed all the positive cases as shown by the false negatives.

```
Final training balanced accuracy: 0.5  
Final training coverage: 1.0
```

**Figure 2.5.3: Final accuracy and coverage**

From the experiment results shown in Figure 2.5.3, the final balanced accuracy was 0.5 and the final coverage was 1.0. A training balanced accuracy of 0.5 tells us that the model could not

distinguish between the two classes in the training data. Because only class 0 was predicted and no class 1 predictions occurred, the equation for balanced accuracy returns 0.5. This means that the model only learned class 0 which was “no heart disease”. The training coverage of 1.0 tells us that all of the training samples matched at least one baseline eLCS rule. However most of the rules predicted class 0, meaning that rules were not effective enough to predicting the minority class being class 1.

## 2.6 Minimal processing required for LCS compatibility

```
# all labels that include heart disease  
HEART_DISEASE_LABELS = [  
    "HT",  
    "DI_HT",  
    "HT_HY",  
    "DI_HT_HY",  
]  
  
# copies the raw data and creates the binary target  
df_baseline = df_raw.copy()  
  
df_baseline["heart_disease_target"] = (  
    df_baseline["sublabel"]  
    .isin(HEART_DISEASE_LABELS)  
    .astype(int)  
)
```

**Figure 2.6.1: Target construction**

Before running the dataset through the baseline eLCS model, we had to make it compatible. Before there was no target column for the model to make predictions into, so we created the heart\_disease\_target column as the target. The sublabel attribute was used to create the target value, which contained combined keys such as HT, DI, HY, and N. HT represented heart disease, so any records with HT in their sublabel were mapped to class 1 (heart disease), and the other records that

didn't contain HT were mapped to class 0 (no heart disease). The true/false values were converted to numeric 1 and 0 because the eLCS model requires numeric values. This was necessary in order for the model to use this dataset, and the original dataset was left untouched, for the sake of backup.

```
# the target-related columns that cannot be predictors
EXCLUDED_COLUMNS = [
    "heart_disease_target",
    "sublabel",
    "disease_flags",
    "heart_disease",
]

# separates the predictor variables from the target
X = df_baseline.drop(columns=EXCLUDED_COLUMNS)
y = df_baseline["heart_disease_target"].astype(int)
```

Figure 2.6.2: Excluded Columns

```
# create a shuffled and stratified 80/20 train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=SEED,
    stratify=y,
    shuffle=True,
)
```

Figure 2.6.3: 80/20 split

Figure 2.6.2 shows how `heart_disease_target`, `sublabel`, `disease_flags`, and `heart_disease` were excluded from being used as predictors. `Heart_disease_target` was separated for being the target, so it can't be the predictor. `Sublabel` was used to construct the target so this also could not be used as it would leak the answer. `Disease_flags` held information about disease which could also leak the answer. Finally `heart_disease` held information about the patient that the eLCS model could use to ignore every other value and guess whether the patient had heart disease or not.

Figure 2.6.3 shows how the 80/20 split was used to create the training and testing sets. Shuffling was used in order to reduce the influence of the original dataset's row order. The `test_size=0.20` makes sure that the test set is 20% of the records and the train set will be 80% of the records. Stratification was also used so that the 2.1% class 1 rate would remain in both sets, this is to keep both sets as similar to the original dataset as possible. Finally a random seed was used so that this split could be reproducible.

```
# creates an encoder that can handle unseen test-set categories
encoder = OrdinalEncoder(
    handle_unknown="use_encoded_value",
    unknown_value=-1,
)

# copies the test/train sets before replacing categorical columns
X_train_encoded = X_train.copy()
X_test_encoded = X_test.copy()

# learns category mappings from the training data and applies them to both partitions.
X_train_encoded[categorical_columns] = encoder.fit_transform(
    X_train[categorical_columns]
)
X_test_encoded[categorical_columns] = encoder.transform(
    X_test[categorical_columns]
)
```

Figure 2.6.4: Category encoding

The categorical attributes in the dataset were converted into numerical values so that they could be processed through the eLCS model. Each categorical value was given a numerical code, and the encoder learned these mappings and applied it to the testing set. Using `fit_transform()` and `transform()` on the training and

testing sets respectively prevented the leakage of data.

### 3. Data Preprocessing and Feature Engineering

- ☐ Students must improve the quality of the dataset before developing the improved system and make it suitable for working with LCS. Students are required to:

- ☒ Address missing values, duplicate records, invalid values, inconsistent categories, and incorrect data types.
- ☒ Investigate and handle outliers where appropriate.
- ☒ Encode categorical variables and scale, discretise, or transform numerical variables where required.
- ☐ Address class imbalance where relevant.
- ☒ Apply feature engineering, feature selection, or dimensionality reduction where justified.
- ☒ Clearly explain how preprocessing is expected to improve LCS performance.
- ☒ Ensure that preprocessing avoids data leakage.

#### 3.1 Data-leakage prevention measures

To prevent target leakage of heart\_disease\_target, there are multiple decisions made during the processing phase to address this issue. Two measures have been used to reduce the data-leakage, Train/test leakage and target leakage.

##### Train/Test leakage

```
Train shape: (222960, 39)
Test shape: (55741, 39)

Train target distribution:
heart_disease_target
False    0.978862
True     0.021138
Name: proportion, dtype: float64

Test target distribution:
heart_disease_target
False    0.978867
True     0.021133
Name: proportion, dtype: float64
```

The train/test split has been used before any other preprocessing measures. This is because splitting the data used for training and model testing should be divided. Models that are based on the same instances of data would cause the model to practically memorise the pattern instead of learning new patterns. A 80/20% split is used when 80% of the data is used for training while 20% is used for testing(The 80/20 split would be further explained in **section 5**).

Additional splitting the testing and training data stops the testing data from influencing the preprocessing phase, e.g calculating the median of glucose, as the testing data would change the median compared to just using the training data.

## Target leakage

These columns label, sublabel, disease\_flags, and heart\_disease are removed in the processing stage because they have partial or full information about the target. The labels column classifies an individual as abnormal or normal with any type of disease, including heart disease, so this cannot be used. For the sublabel column, this was used to directly create the target because it contains the classification of diseases. Disease\_flags column contains the binary indicator of which disease the individual has. Lastly, the heart\_disease column shows the probability of heart disease for a patient with similar traits and showed a strong correlation of 0.65 with the target; therefore, it cannot be used.

```
columns_to_drop = ["sublabel", "disease_flags", "heart_disease", "label"] # All the columns to drop because it contains information about the target

X_train = X_train.drop(columns=columns_to_drop)
X_test = X_test.drop(columns=columns_to_drop)

print(X_train.columns.tolist())
✓ 0.0s
['composite_key', 'age_level', 'gender', 'bmi_level', 'smoking', 'diabetes', 'age', 'age_normalized', 'bmi', 'hypertension', 'HbA1c_level', 'glucose', 'c
```

(Note: **The rest of the processing is completed on the training dataset and then applied to the testing dataset**)

## 3.2 Data cleaning

In phase 1, the raw dataset has been cleaned for exploratory analysis (including missing values, duplicate records, inconsistent categories, and data types). Before going further, the dataset has been reviewed again to check if there are any issues after the splitting of the training and testing datasets. Invalid values were partially checked during phase 1 EDA; therefore, we will review them again.

### Missing Values

```
Missing values in X_train: 0
Missing values in X_test: 0
```

Checked the missing values in the training set testing sets. No missing values were found. This is to ensure that the cleaning completed in phase 1 was

inspected again.

## Duplicate Records

```
Duplicate rows in X_train: 177
Duplicate rows in X_test: 8
Target distribution among duplicate rows:
heart_disease_target
True      177
False     177
Name: count, dtype: int64

Example of conflicting duplicate pairs:
```

|       | composite_key                       | age_level | gender | bmi_level  | smoking | diabetes | age | age_normalized | bmi   | hypertension | ... | systolic_bp | diastolic_bp | alcohol_intake | salt_intake |
|-------|-------------------------------------|-----------|--------|------------|---------|----------|-----|----------------|-------|--------------|-----|-------------|--------------|----------------|-------------|
| 25433 | Mature_Female_Overweight_Current_No | Mature    | Female | Overweight | Current | No       | 56  | 0.620690       | 27.32 | 0.0          | ... | 134.235967  | 89.338877    | 15.049688      | 8.27474     |
| 46715 | Mature_Female_Overweight_Current_No | Mature    | Female | Overweight | Current | No       | 56  | 0.620690       | 27.32 | 0.0          | ... | 134.235967  | 89.338877    | 15.049688      | 8.27474     |
| 48110 | Mature_Female_Overweight_Former_No  | Mature    | Female | Overweight | Former  | No       | 51  | 0.563218       | 27.32 | 0.0          | ... | 134.117709  | 89.586426    | 14.855673      | 8.50859     |
| 26464 | Mature_Female_Overweight_Former_No  | Mature    | Female | Overweight | Former  | No       | 51  | 0.563218       | 27.32 | 0.0          | ... | 134.117709  | 89.586426    | 14.855673      | 8.50859     |

When checking for duplicate records after taking measures for data leakage, removing the columns that have information about the target introduces 177 duplicate records in the training dataset and 8 records in the testing dataset. This is because the target leakage columns contain values that make these instances unique. If the data were truly duplicated, then the target should not have conflicting results when the other columns have identical values. Therefore, these instances could not be removed since the different target values imply that these instances are different.

## Invalid Values

|       | age           | bmi           | glucose       | cholesterol   |
|-------|---------------|---------------|---------------|---------------|
| count | 222960.000000 | 222960.000000 | 222960.000000 | 222960.000000 |
| mean  | 49.725368     | 27.490879     | 135.266042    | 223.985144    |
| std   | 21.682929     | 7.067417      | 38.471530     | 35.681088     |
| min   | 2.000000      | 10.010000     | 70.000000     | 150.000000    |
| 25%   | 32.000000     | 22.000000     | 100.000000    | 201.000000    |
| 50%   | 50.000000     | 27.320000     | 137.000000    | 225.000000    |
| 75%   | 68.000000     | 32.700000     | 160.000000    | 243.000000    |
| max   | 89.000000     | 95.690000     | 300.000000    | 300.000000    |

|       | systolic_bp   | diastolic_bp  | heart_rate    |
|-------|---------------|---------------|---------------|
| count | 222960.000000 | 222960.000000 | 222960.000000 |
| mean  | 134.420094    | 89.493606     | 74.505100     |
| std   | 20.606393     | 13.720323     | 11.450341     |
| min   | 90.000000     | 60.000000     | 50.000000     |
| 25%   | 125.000000    | 83.000000     | 69.000000     |
| 50%   | 134.167413    | 89.504587     | 74.487247     |
| 75%   | 144.000000    | 96.000000     | 80.000000     |
| max   | 179.000000    | 119.000000    | 99.000000     |

Invalid values were checked by making sure that the values are within valid boundaries. Age, BMI, glucose, cholesterol, systolic\_bp, diastolic\_bp, and heart\_rate all showed that the values are plausible.

## Inconsistent categories

```

composite_key:
composite_key
Middle_Female_Overweight_Never_No    4441
Mature_Female_Overweight_Never_No    4302
Mature_Female_Obese_Never_No         4029
Middle_Female_Obese_Never_No         4011
Middle_Female_Normal_Never_No        3853
...
Young_Female_Underweight_Current_No  148
Young_Female_Underweight_Never_Yes   148
Young_Male_Underweight_Former_Yes     138
Young_Male_Underweight_Current_Yes    133
Young_Female_Underweight_Former_Yes   119
Name: count, Length: 192, dtype: int64

age_level:
age_level
Mature    69654
Middle    67574
Senior    55874
Young     29858
Name: count, dtype: int64

gender:
...
hypertension    139929
diabetes         75162
heart            7869
Name: count, dtype: int64
Output is truncated. View as a scrollable element or open in a text

```

All categorical columns were checked for inconsistent categories. Composite key, age\_level, gender, bmi\_level, smoking, diabetes, physical\_activity, family\_history, stress\_level, low\_hdl\_cholesterol, high\_ldl\_cholesterol, high\_blood\_pressure, sugar\_consumption, education\_level, employment\_status and source\_dataset, all have consistent categorical values and formatting consistency.

## Data Types

```

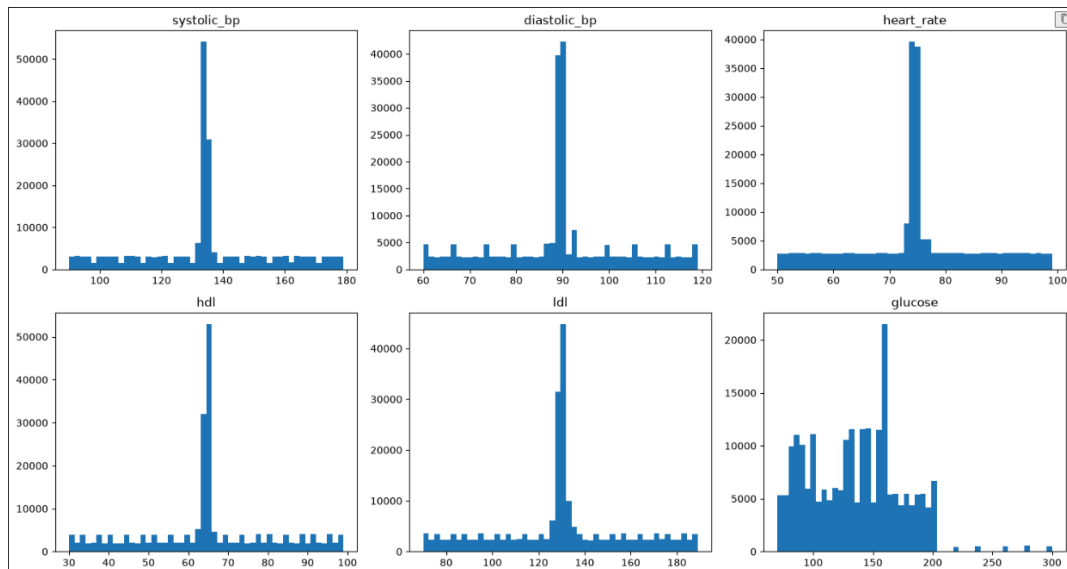
composite_key    str
age_level        str
gender           str
bmi_level        str
smoking          str
diabetes         str
age              int64
age_normalized   float64
bmi              float64
hypertension     float64
HbA1c_level      float64
glucose          float64
cholesterol      float64
sleep_hours      float64
triglycerides    float64
physical_activity str
family_history    str
stress_level     str
low_hdl_cholesterol str
high_ldl_cholesterol str
blood_pressure   float64
high_blood_pressure str
sugar_consumption str
crp_level        float64
homocysteine_level float64
...
education_level  str
employment_status str
source_dataset   str
dtype: object

```

The data types of each column are correct; therefore, no changes would be made.

### 3.3 Outlier investigation and handling

```
age: lower=-22.00, upper=122.00, outliers=0
bmi: lower=5.95, upper=48.75, outliers=788
glucose: lower=10.00, upper=250.00, outliers=1624
cholesterol: lower=138.00, upper=306.00, outliers=0
triglycerides: lower=-42.62, upper=412.38, outliers=0
systolic_bp: lower=96.50, upper=172.50, outliers=21854
diastolic_bp: lower=63.50, upper=115.50, outliers=18579
heart_rate: lower=52.50, upper=96.50, outliers=16665
hdl: lower=37.00, upper=93.00, outliers=25988
ldl: lower=79.50, upper=179.50, outliers=23413
```



In phase 1, BMI was found to have a considerable number of outliers, but it is proven that the outliers shouldn't be transformed or removed. The other columns were discussed at the surface level, based on the boxplot, and showed no isolated instances. To further investigate this, the Interquartile range method was performed on the numerical columns. Triglyceride, cholesterol, and age showed no outliers, but the rest of the columns' results displayed incredibly high values, e.g. 18579 outliers for diastolic\_bp as well as other columns. A histogram is created to show the distribution of instances across each column. Most of the columns (systolic\_bp, diastolic\_bp, heart\_rate, hdl, ldl, glucose) showed a high peak in the middle and consistent values across the whole range. This is why using the IQR method and boxplot isn't enough to test for outliers and to show the outliers accurately. IQR is used only when the data shows a normal distribution; in this case, the data is not. It is concluded that these outliers would be treated as valid instances; thus, no removal, transformation, or capping is performed.

### 3.4 Feature engineering

In the current dataset up to this point of processing, multiple columns are irrelevant or provide no value towards predicting the target.

- Composite key (**removed**)

A composite key is just the combination of variables (age\_level, gender, bmi\_level, smoking, and diabetes); therefore, this does not provide any value, as it is simplifying group the variables for the instance. So removing this would not cause any information to be lost and is just an unnecessary column to provide to the model.

- Age\_level and age\_normalized (**removed**)

Age\_level and age\_normalized provide similar information to age, but age\_level is a categorical level and age\_normalized is the linear scaling of age and showed a 1.00 correlation with age during EDA. These add no additional information about age, as age could be discretised independently for eLCS, thus being a more favourable column to remain in the training dataset.

- bmi (**removed**)

The bmi column is removed because the bmi\_level column follows the WHO standard, which is the standard classifier for BMI. The bmi column does allow independent interpretation to discretization, but since bmi\_level is already in a suitable standard, there isn't any reason not to use bmi\_level instead of bmi.

### 3.5 Encoding categorical variables

```
X_train[encoded_cols].head().T
```

|                              | 226737 | 240229 | 263800 | 102871 | 209771 |
|------------------------------|--------|--------|--------|--------|--------|
| gender                       | 1.0    | 0.0    | 1.0    | 1.0    | 0.0    |
| diabetes                     | 1.0    | 0.0    | 0.0    | 1.0    | 1.0    |
| family_history               | 0.0    | 1.0    | 0.0    | 1.0    | 0.0    |
| low_hdl_cholesterol          | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    |
| high_hdl_cholesterol         | 0.0    | 1.0    | 0.0    | 1.0    | 0.0    |
| high_blood_pressure          | 0.0    | 0.0    | 1.0    | 0.0    | 1.0    |
| bmi_level                    | 3.0    | 0.0    | 2.0    | 3.0    | 0.0    |
| smoking                      | 0.0    | 1.0    | 2.0    | 2.0    | 0.0    |
| physical_activity            | 1.0    | 0.0    | 0.0    | 1.0    | 1.0    |
| stress_level                 | 0.0    | 1.0    | 1.0    | 2.0    | 1.0    |
| sugar_consumption            | 1.0    | 2.0    | 2.0    | 2.0    | 1.0    |
| education_level              | 1.0    | 0.0    | 0.0    | 2.0    | 0.0    |
| employment_status_Employed   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    |
| employment_status_Retired    | 0.0    | 0.0    | 1.0    | 0.0    | 0.0    |
| employment_status_Unemployed | 1.0    | 1.0    | 0.0    | 1.0    | 1.0    |
| source_dataset_diabetes      | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    |
| source_dataset_heart         | 0.0    | 0.0    | 0.0    | 1.0    | 0.0    |
| source_dataset_hypertension  | 1.0    | 1.0    | 1.0    | 0.0    | 1.0    |

All the categorical variables have been encoded to numerical values because eLCS could not accept values in strings. For example, gender has been changed from “Female” and “Male” to 0 and 1.

Multiple types of encoding have been used for the categorical variables since some variables required different encoding methods.

For these columns (gender, diabetes, family\_history, low\_hdl\_cholesterol, high\_ldl\_cholesterol, and high\_blood\_pressure), binary encoding is used because these columns only have 2 possible outputs.

For these columns (bmi\_level, smoking, physical\_activity, stress\_level, sugar\_consumption, "education\_level), ordinal encoding is used because, for example, for stress\_level, there is an order of the output values "low", "medium," and "high"; therefore, the encoding converts it into numerical values (1 to 3).

For the rest of the categorical columns ("employment\_status", "source\_dataset"), one-hot encoding is used since they provide no order and cannot be represented in binary; therefore, new columns would be created to represent each possible outcome, as shown in the encoding categorical screenshot.

### 3.6 Discretising & Encoding numerical variables

|        | glucose_binned       | cholesterol_binned | systolic_bp_binned        | \                 |   |
|--------|----------------------|--------------------|---------------------------|-------------------|---|
| 226737 | 2.0                  | 0.0                | 3.0                       |                   |   |
| 240229 | 2.0                  | 0.0                | 2.0                       |                   |   |
| 263800 | 2.0                  | 0.0                | 0.0                       |                   |   |
| 102871 | 2.0                  | 1.0                | 2.0                       |                   |   |
| 209771 | 0.0                  | 2.0                | 3.0                       |                   |   |
|        | diastolic_bp_binned  | HbA1c_level_binned | age_binned                | \                 |   |
| 226737 | 2.0                  | 2.0                | 3.0                       |                   |   |
| 240229 | 0.0                  | 0.0                | 1.0                       |                   |   |
| 263800 | 2.0                  | 0.0                | 2.0                       |                   |   |
| 102871 | 1.0                  | 2.0                | 2.0                       |                   |   |
| 209771 | 1.0                  | 2.0                | 1.0                       |                   |   |
|        | triglycerides_binned | hdl_binned         | ldl_binned                | heart_rate_binned | \ |
| 226737 | 0.0                  | 2.0                | 1.0                       | 1.0               |   |
| 240229 | 0.0                  | 2.0                | 3.0                       | 1.0               |   |
| 263800 | 2.0                  | 2.0                | 0.0                       | 1.0               |   |
| 102871 | 2.0                  | 2.0                | 2.0                       | 1.0               |   |
| 209771 | 2.0                  | 2.0                | 0.0                       | 1.0               |   |
|        | crp_level_binned     | sleep_hours_binned | homocysteine_level_binned | \                 |   |
| 226737 | 2.0                  | 0.0                | 0.0                       |                   |   |
| 240229 | 2.0                  | 0.0                | 0.0                       |                   |   |
| 263800 | 2.0                  | 1.0                | 0.0                       |                   |   |
| ...    |                      |                    |                           |                   |   |
| 240229 | 0.0                  |                    | 0.0                       | 0.0               |   |
| 263800 | 0.0                  |                    | 2.0                       | 1.0               |   |
| 102871 | 1.0                  |                    | 1.0                       | 1.0               |   |
| 209771 | 0.0                  |                    | 0.0                       | 2.0               |   |

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Discretising has been used for all the numerical datasets to turn them into categorical values because eLCS operates on if-then rules, which requires categorical data in order for the prediction to happen.

To determine the categorical values, the columns that have clinical standards are then used to determine whether they are low, medium, or high. For example, for glucose, the normal range would be below or at 100mg/dL, the prediabetes level at 100 to 126mg/dL and diabetes at 126mg/dL and above. The columns (hypertension, alcohol\_intake and salt\_intake)

that don't have a set standard of measuring levels require the use of quantile-based discretisation. This is to roughly split all training data into 3 categorical values based on their relative numerical value to each other on each column. For example, hypertension in this dataset is a continuous range from 0-1. There is no standard measurement for this since hypertension is standardly measured in mmHg.

After discretising all the numerical data, these are then encoded so that eLCS could utilize them in the prediction.

### **3.7 Class-imbalance handling**

In the EDA, the target variable is highly imbalanced across the dataset, with 2.1% of instances with heart disease and 97.9% of instances without heart disease of the total 278,701 instances. The baseline eLCS prediction results showed that the final balanced accuracy test is 0.5; this effectively chooses negative (no heart disease) for all the instances. This result is no better than just predicting half of the instance does and half the instance does not when considering the balanced accuracy.

The imbalance would be handled after receiving the result from the prediction preprocessed dataset(**Task 4**) in eLCS. This is because it would be valuable insight into how much class imbalance could affect the overall prediction of the model when the imbalance is handled.

### **3.8 Expected impact of preprocessing on LCS performance**

The overall preprocessing phase of the dataset is expected to improve the eLCS ability to process and performance, these improvements include:

- **Improved Computational cost / quicker prediction**

In the feature selection process, multiple redundant columns were removed, therefore there are less combinations of if and then rules.

- **More compact rule set**

Applying the discretization lowers the amount of classifiers since there will be less classifiers that are tied to continuous values which also increase readability for validation and inspection of the rule.

- **Improved accuracy of rules**

Using the appropriate encoding types would reduce the number of misleading information when just applying a general encoding type to all columns which causes the eLCS to learn inaccurate patterns.

- **Increase trustworthiness of prediction**

This is because the columns that are removed that provide information about the target information would influence eLCS ability to learn and accurately predict the target without genuinely learning from the other features.

## 4. Improved LCS-Based System

☐ Students must improve the LCS code, LCS configuration, or machine learning pipeline to develop a novel improved system. Possible improvements include:

- ☐ Hyperparameter tuning.
- ☐ Improved preprocessing designed for LCS performance.
- ☐ Feature selection or dimensionality reduction.
- ☐ Rule pruning or rule compaction.
- ☐ Modified LCS algorithm.
- ☒ ~~Class imbalance handling.~~
- ☐ Cost-sensitive learning.

☐ Students are required to:

- ☒ ~~Clearly identify what was changed compared with the original LCS system.~~
- ☒ ~~Justify why the change may improve prediction accuracy or model quality.~~
- ☒ ~~Provide a brief algorithmic explanation, pseudocode, or flowchart of the improved system.~~
- ☐ Compare the improved system with the original LCS baseline.

## 4.1 Summary of changes vs. the original LCS system

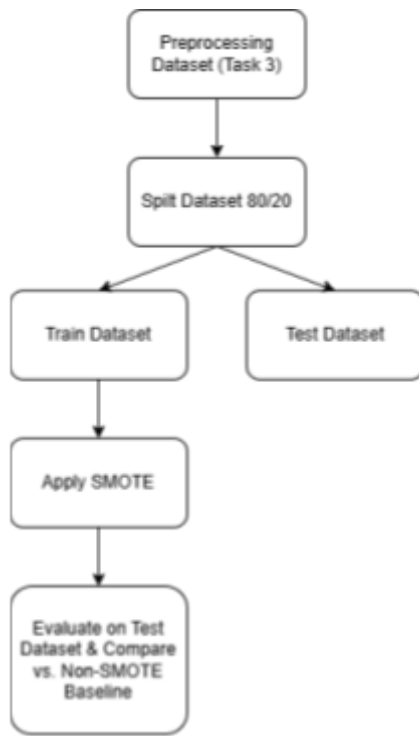
Regarding the changes from the original system, the preprocessing in **task 3** compared to the baseline LCS model in **task 2**, the preprocessing stage included an attempt to remove data leakage, improve the quality and accuracy of the data (missing values, duplicate records, invalid values, inconsistent categories, data types, and outliers), and perform transformations (feature selection, encoding, and discretization).

A novel improvement introduced in **task 4** compared to the preprocessing stage is addressing the class imbalance using SMOTE (Synthetic Minority Oversampling Technique). This is used to address the heavy class imbalance of the 2.1% of instances with heart disease. This is the only novel improvement that would be made, since having just one change would allow a comparison of before addressing the class imbalance and after addressing the class imbalance, to be compared since especially for this dataset class imbalance is a massive factor. Multiple improvements made would make it hard to identify which change has a change in the model results. The ELCS hyperparameters would not be changed in this stage.

## 4.2 Justification

Instead of using basic imbalance techniques like oversampling or undersampling, implementing the SMOTE technique would create artificial instances based on interpreting the current minority class; this adds variation towards the heart disease class instance instead of duplicating the current heart disease instances. As well as instead of using undersampling, which reduces the amount of the majority class to match the minority class, since 97.9% is the majority class, this would drastically remove a significant amount of the instances. The only issue is that the majority of the minority class would be artificially created instead.

### 4.3 Pseudocode of the improved system



This pipeline shows the step-by-step process of how the novel improvement is implemented towards the existing preprocessing eLCS and the evaluation with the resulting model.

Figure 4.3.1 FlowChart of improved system

### 4.4 Comparison against the original LCS baseline

Arrays ready. X\_train\_array shape: (222960, 35)  
 No SHOTE - 10 runs:

|   | Seed | Accuracy | Precision | Recall   | F1       | Balanced accuracy | ROC-AUC  | PR-AUC   | Training seconds | True negatives | False positives | False negatives | True positives |
|---|------|----------|-----------|----------|----------|-------------------|----------|----------|------------------|----------------|-----------------|-----------------|----------------|
| 0 | 3    | 0.978867 | 0.000000  | 0.000000 | 0.000000 | 0.500000          | 0.623093 | 0.049519 | 46.606339        | 54563          | 0               | 1178            | 0              |
| 1 | 6    | 0.978867 | 0.000000  | 0.000000 | 0.000000 | 0.500000          | 0.644734 | 0.059376 | 37.705084        | 54563          | 0               | 1178            | 0              |
| 2 | 9    | 0.978795 | 0.300000  | 0.002547 | 0.005051 | 0.501209          | 0.597073 | 0.047600 | 38.149765        | 54556          | 7               | 1175            | 3              |
| 3 | 12   | 0.978813 | 0.000000  | 0.000000 | 0.000000 | 0.499973          | 0.570183 | 0.034534 | 37.867776        | 54560          | 3               | 1178            | 0              |
| 4 | 15   | 0.978867 | 0.000000  | 0.000000 | 0.000000 | 0.500000          | 0.562215 | 0.042381 | 38.193331        | 54563          | 0               | 1178            | 0              |
| 5 | 18   | 0.978867 | 0.000000  | 0.000000 | 0.000000 | 0.500000          | 0.553553 | 0.039653 | 36.964514        | 54563          | 0               | 1178            | 0              |
| 6 | 21   | 0.978867 | 0.000000  | 0.000000 | 0.000000 | 0.500000          | 0.575517 | 0.045619 | 38.334572        | 54563          | 0               | 1178            | 0              |
| 7 | 24   | 0.978867 | 0.500000  | 0.000849 | 0.001695 | 0.500415          | 0.611364 | 0.064501 | 37.987052        | 54562          | 1               | 1177            | 1              |
| 8 | 27   | 0.978490 | 0.200000  | 0.005942 | 0.011542 | 0.502715          | 0.577628 | 0.044593 | 36.887387        | 54535          | 28              | 1171            | 7              |
| 9 | 30   | 0.978185 | 0.104167  | 0.004244 | 0.008157 | 0.501728          | 0.562085 | 0.036606 | 37.665084        | 54520          | 43              | 1173            | 5              |

With SHOTE - 10 runs:

|   | Seed | Accuracy | Precision | Recall   | F1       | Balanced accuracy | ROC-AUC  | PR-AUC   | Training seconds | True negatives | False positives | False negatives | True positives |
|---|------|----------|-----------|----------|----------|-------------------|----------|----------|------------------|----------------|-----------------|-----------------|----------------|
| 0 | 3    | 0.814714 | 0.100506  | 0.977080 | 0.182264 | 0.894144          | 0.931587 | 0.159612 | 82.404875        | 44262          | 10301           | 27              | 1151           |
| 1 | 6    | 0.791356 | 0.091016  | 0.987267 | 0.166667 | 0.887197          | 0.898956 | 0.099536 | 84.931392        | 42948          | 11615           | 15              | 1163           |
| 2 | 9    | 0.808992 | 0.097509  | 0.973684 | 0.177266 | 0.889560          | 0.929419 | 0.150306 | 1687.054069      | 43947          | 10616           | 31              | 1147           |
| 3 | 12   | 0.784180 | 0.088128  | 0.985569 | 0.161789 | 0.882701          | 0.932236 | 0.156425 | 83.899419        | 42550          | 12013           | 17              | 1161           |
| 4 | 15   | 0.786997 | 0.088938  | 0.982173 | 0.163107 | 0.882478          | 0.927689 | 0.155017 | 84.315346        | 42711          | 11852           | 21              | 1157           |
| 5 | 18   | 0.825209 | 0.104753  | 0.963497 | 0.188962 | 0.892861          | 0.936552 | 0.160211 | 80.947682        | 44863          | 9700            | 43              | 1135           |
| 6 | 21   | 0.792379 | 0.091359  | 0.986418 | 0.167230 | 0.887304          | 0.922829 | 0.132226 | 87.119622        | 43006          | 11557           | 16              | 1162           |
| 7 | 24   | 0.786638 | 0.089180  | 0.987267 | 0.163584 | 0.884787          | 0.926254 | 0.142481 | 87.745493        | 42685          | 11878           | 15              | 1163           |
| 8 | 27   | 0.823559 | 0.102926  | 0.952462 | 0.185777 | 0.886619          | 0.933953 | 0.157026 | 82.487264        | 44784          | 9779            | 56              | 1122           |
| 9 | 30   | 0.803412 | 0.094998  | 0.973684 | 0.173106 | 0.886710          | 0.934016 | 0.159611 | 81.282653        | 43636          | 10927           | 31              | 1147           |

Figure 4.4.t: Individual values of prediction training

|   | Metric               | mean      | std          | min       | max       |
|---|----------------------|-----------|--------------|-----------|-----------|
| 0 | Accuracy             | 0.978985  | 1.170278e-16 | 0.978985  | 0.978985  |
| 1 | Precision            | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 2 | Recall               | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 3 | F1                   | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 4 | Balanced accuracy    | 0.500000  | 0.000000e+00 | 0.500000  | 0.500000  |
| 5 | ROC-AUC              | 0.580983  | 3.751011e-02 | 0.522048  | 0.649815  |
| 6 | PR-AUC               | 0.044048  | 7.779176e-03 | 0.034085  | 0.055804  |
| 7 | Positive predictions | 0.000000  | 0.000000e+00 | 0.000000  | 0.000000  |
| 8 | Training seconds     | 12.517976 | 2.214622e-01 | 12.252371 | 12.864080 |

BaseLine Results

No SMOTE summary (10 runs):

|   | Metric            | mean      | std      | min       | max       |
|---|-------------------|-----------|----------|-----------|-----------|
| 0 | Accuracy          | 0.978748  | 0.000230 | 0.978185  | 0.978867  |
| 1 | Precision         | 0.110417  | 0.172862 | 0.000000  | 0.500000  |
| 2 | Recall            | 0.001358  | 0.002162 | 0.000000  | 0.005942  |
| 3 | F1                | 0.002644  | 0.004192 | 0.000000  | 0.011542  |
| 4 | Balanced accuracy | 0.500604  | 0.000963 | 0.499973  | 0.502715  |
| 5 | ROC-AUC           | 0.587744  | 0.030151 | 0.553553  | 0.644734  |
| 6 | PR-AUC            | 0.046438  | 0.009484 | 0.034534  | 0.064501  |
| 7 | Training seconds  | 38.636091 | 2.842566 | 36.887387 | 46.606339 |

Preprocessing Results

With SMOTE summary (10 runs):

|   | Metric            | mean       | std        | min       | max         |
|---|-------------------|------------|------------|-----------|-------------|
| 0 | Accuracy          | 0.801744   | 0.015634   | 0.784180  | 0.825209    |
| 1 | Precision         | 0.094931   | 0.006160   | 0.088128  | 0.104753    |
| 2 | Recall            | 0.976910   | 0.011576   | 0.952462  | 0.987267    |
| 3 | F1                | 0.172975   | 0.010043   | 0.161789  | 0.188962    |
| 4 | Balanced accuracy | 0.887436   | 0.003862   | 0.882478  | 0.894144    |
| 5 | ROC-AUC           | 0.927349   | 0.010785   | 0.898956  | 0.936552    |
| 6 | PR-AUC            | 0.147245   | 0.018997   | 0.099536  | 0.160211    |
| 7 | Training seconds  | 244.218781 | 506.965706 | 80.947682 | 1687.054069 |

Preprocessing with SMOTE Results

**Figure 4.4.2 Metric Summary**

The result of the implementation of SMOTE (on training) displayed a massive difference between the preprocessed training and the baseline. Across the mean of the 10 seeds used, the baseline and the preprocessed result (precision, recall, F1, balanced accuracy, ROC-AUC, and PR-AUC) there is virtually no difference since the recall, F1, and balanced accuracy are similar. Additionally, the baseline predicted no positive cases, while the preprocessed model predicted a couple of positive cases, but this is too few to be considered reliable. Therefore, just the comparison between the preprocessed model and the

preprocessed model with SMOTE is more meaningful than comparing all training models. With SMOTE testing metrics, the accuracy decreased from 0.98 to 0.80 mean (in preprocessing) since the model predicts the positive class more often. As shown in Figure 4.4.1, the confusion matrix showed a considerable amount more of true positives and false positives (around 9700-12000). Since nearly 98% of the test dataset is negative, a small percentage of misclassified instances would result in thousands of false positives. Precision decreased from 0.11 (preprocessing) to 0.09 (SMOTE); this decrease is also due to the increase in false positives, and also the reason why the precision is higher for preprocessing is that there are fewer positive predictions. Recall had a massive difference, from 0.001 (preprocessing) to 0.97 (SMOTE); this is due to the training model successfully predicting most of the positive instances.

Using SMOTE to solve class imbalance is overall valid because the F1, balanced accuracy, ROC-AUC, and PR-AUC give the fairest visualization of the impact of solving class imbalance. From 0.5 balanced accuracy to 0.88, which means that the model is able to differentiate the two different classes more easily than the preprocessing model. The other statistical metrics have increased (F1 from 0.003 to 0.173 and PR-AUC from 0.046 to 0.147), but they are still fairly low since these metrics are heavily relevant on the precision metric. These statistical metrics would be further analyzed in task 5. The other limitation is that this introduces a significantly longer training time, from a mean of 38 seconds to 244 seconds, because of the addition of artificially created positive instances from the use of SMOTE.

## 5. Experiments

- ☐ Students must use a suitable and justified validation strategy. Students are required to:
  - ☐ Use either an 80/20 train-test split or k-fold cross-validation.
  - ☐ Justify the selected validation approach.
- ☐ Students must evaluate results using appropriate metrics and statistical tests. Students are required to:
  - ☐ Select evaluation metrics suitable for the selected problem.
  - ☐ For classification, consider accuracy, precision, recall, F1-score, balanced accuracy, ROC-AUC, PR-AUC, and confusion matrix.
  - ☐ For regression, consider MAE, MSE, RMSE, MAPE,  $R^2$ , and residual analysis.
  - ☐ Justify the selected metrics.
  - ☐ Apply at least one suitable statistical test to compare model performance.
  - ☐ Interpret whether performance differences are statistically significant.

### 5.1 Validation strategy (80/20 split or k-fold CV) – justification

We went with a 80/20 split which split the dataset into a training set with 80% of the data, and a testing set with 20% of the data. In the original dataset, approximately 2.1% of patients had heart-disease. This is a very low number and introduces class imbalance to the dataset. To keep this environment, stratification was used to keep the same 2.1% proportion, making the training and testing sets similar to the raw dataset. A seed was recorded and used for the split so that the 80/20 split could be

reproducible, just in case we wanted to re run the split again with the same results. The exact same seed was used in the baseline model to make sure the comparison between the baseline eLCS model, and the improved model was fair.

The 80/20 split was a better approach rather than k-fold cross validation, mainly because of the size of the dataset. and the computational requirements. K-fold CV requires the model to be trained multiple times with k representing the number of times the model will be trained. The eLCS model keeps on creating, evaluating, and evolving rules in each training, so this whole process happening repeatedly is very computationally costly. The dataset contains over 200k rows which is more than enough for an 80/20 split to be effective for machine learning. If the dataset were smaller, then it would be less reliable because the training test would not contain enough data to train the model.

## **5.2 Evaluation metrics – selection and justification (classification: accuracy, precision, recall, F1, balanced accuracy, ROC-AUC, PR-AUC, confusion matrix; regression: MAE, MSE, RMSE, MAPE, $R^2$ , residual analysis)**

## **5.3 Statistical significance testing**

## **5.4 Interpretation of significance results**

# **6. Model Comparison**

- ☐ Students must compare the proposed improved LCS-based system with the original LCS and at least three conventional machine learning models. The comparison must include:
  - ☐ Original LCS on the raw or minimally processed dataset.
  - ☐ Original LCS on the preprocessed dataset.
  - ☐ Improved LCS-based system on the preprocessed and/or feature-engineered dataset.
  - ☐ At least three conventional non-deep-learning machine learning models.
- ☐ Conventional models may include Random Forest, Support Vector Machine, Decision Tree, Logistic Regression, Naive Bayes, k-Nearest Neighbours, Gaussian Process, or other appropriate non-deep-learning models. Deep-learning models are not permitted for the required model comparison.

**6.1 Original LCS – raw/minimally processed data**

**6.2 Original LCS – preprocessed data**

**6.3 Improved LCS – preprocessed/feature-engineered data**

**6.4 At least three conventional ML models (e.g. Random Forest, SVM, Decision Tree, Logistic Regression, Naive Bayes, k-NN, Gaussian Process) – no deep learning**

**6.5 Comparative results table**

## **7. Interpretation of Results and Explainable AI**

- ☐ Students must include a separate section on result interpretation and Explainable AI. Students are required to:
  - ☐ Interpret the main findings from the model comparison.
  - ☐ Explain important LCS rules, feature conditions, or classifier patterns.
  - ☐ Use at least three sample LCS rules/classifiers to discuss the decision-making process.
  - ☐ Discuss how LCS rule-based outputs support interpretability.
  - ☐ Discuss whether the explanations are trustworthy and practically meaningful.

**7.1 Interpretation of main findings from the model comparison**

**7.2 Explanation of key LCS rules / feature conditions / classifier patterns**

**7.3 At least three sample LCS rules discussed for decision-making insight**

**7.4 How rule-based outputs support interpretability**

**7.5 Trustworthiness and practical meaningfulness of the explanations**

## **8. Discussion, Limitations, and Future Work**

☐ Students must critically discuss the results and the responsible use of the developed system. Students are required to discuss:

- ☐ Reasons for performance improvement or lack of improvement.
- ☐ Practical meaning of the results, not only numerical performance.
- ☐ Limitations due to data quality, dataset size, class imbalance, noise, missingness, or feature relevance.
- ☐ Future work, such as improved LCS variants, better preprocessing, external validation, or domain expert review.

**8.1 Reasons for performance improvement (or lack thereof)**

**8.2 Practical significance of results beyond raw numbers**

**8.3 Limitations – data quality, dataset size, class imbalance, noise, missingness, feature relevance**

**8.4 Future work – improved LCS variants, better preprocessing, external validation, domain expert review**

## Notes

Final report: 5,000--6,500 words, excluding references and appendices.

Source code or Jupyter Notebook.

GitHub repository link and commit-history evidence.

Contribution statement, including a brief individual reflection from each group member.

Cleaned and feature-engineered dataset, or clear dataset reproduction instructions.

Turnitin similarity report with a similarity score not exceeding 20%.

## References

### Dataset

- <https://www.kaggle.com/datasets/danishjmeo/281k-patient-dataset-for-multi-disease-prediction?resource=download>

### Sub Datasets

- <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset>,
- [kaggle.com/datasets/oktayrdeki/heart-disease](https://www.kaggle.com/datasets/oktayrdeki/heart-disease),
- [kaggle.com/datasets/ankushpanday1/hypertension-risk-prediction-dataset](https://www.kaggle.com/datasets/ankushpanday1/hypertension-risk-prediction-dataset)

### Research

Latest heart disease statistics. (n.d.). Heart Foundation NZ.

<https://www.heartfoundation.org.nz/statistics>